

title: 病种分析课件\_模块一：破冰与基石 type: 方法论 tags: [DRG, DIP, CMI, 效率分析, 变异系数, 培训, 医保支付]

# 模块一：破冰与基石 —— 读懂医保支付与核心指标

## 导言

本模块作为整个病种分析课程的开篇与基石，旨在建立对现代医保支付体系（DRG/DIP）及其伴生核心指标的深刻理解。我们不仅要知其然，更要知其所以然——从历史沿革、底层算法到统计学原理，全方位拆解这些主导当前医院运营管理的核心概念。只有透彻掌握了这些“游戏规则”与“度量衡”，后续的专科经营分析、绩效考核与临床路径优化才具有坚实的基础。

## 1.1 医保支付改革底层逻辑：DRG与DIP的算法解构

### 1.1.1 历史沿革与原理溯源

传统医保支付方式以“按项目付费”（Fee-For-Service, FFS）为主，其核心特征是“多劳多得”。这种模式极易诱发供给方诱导需求（Supplier-induced Demand），导致过度医疗、大处方和医疗费用螺旋式上升。为控制医疗费用不合理增长，全球范围内逐渐转向前瞻性支付制度（Prospective Payment System, PPS），其中最著名的便是按疾病诊断相关分组（Diagnosis Related Groups, DRG）付费。

**DRG（疾病诊断相关分组）** DRG起源于20世纪70年代末的美国耶鲁大学，最初作为一种衡量医疗服务产出的病例组合（Case-mix）工具。其核心理念是：将临床特征相似、医疗资源消耗相近的病例归为同一组。这实质上是一种“统计学上的聚类”。DRG的分组逻辑是一种自顶向下的“树状决策”过程：

- 主要诊断大类（MDC, Major Diagnostic Categories）**：通常根据解剖系统或病因划分（如MDC A：神经系统疾病及疾患）。这基于主要诊断（Primary Diagnosis）。
- 核心疾病诊断相关组（ADRG, Adjacent DRG）**：在MDC内部，根据是否手术、内科/外科/操作进行粗分，再结合具体诊断和手术操作进行细分。
- 细分DRG**：在ADRG基础上，引入年龄、合并症与并发症（CC/MCC）等其他变量，最终形成DRG组（如伴有严重并发症、伴有一版并发症、不伴并发症）。

**DIP（按病种分值付费）** DIP（Diagnosis-Intervention Packet）是中国本土化创新的支付方式。与DRG的专家共识引导、自顶向下的粗颗粒度聚类不同，DIP是自底向上、基于大数据的细颗粒度枚举。DIP的核心逻辑是：将“主诊断+主操作”作为一个组合（Packet），基于历史真实世界的医疗费用数据，计算该组合的平均费用，并将其转化为“分值”。这是一种基于大数定律和均值回归的定价策略。

### 1.1.2 数学公式与定价模型

**DRG支付公式：** 
$$\text{DRG组支付标准} = \text{该DRG组相对权重(RW)} \times \text{区域基础费率(Base Rate)}$$
 其中， 
$$\text{RW} = \frac{\text{该DRG组内病例的平均费用}}{\text{全样本所有病例的平均费用}}$$

**DIP支付公式：** 
$$\text{DIP病种支付标准} = \text{该病种分值} \times \text{点值(Point Value)}$$
 其中， 
$$\text{病种分值} = \frac{\text{该病种历史平均费用}}{\text{全样本基准病种历史平均费用(或全样本平均费用)}}$$

两者的本质殊途同归：都是以一个基准价格（费率/点值）乘以一个标化难度系数（权重/分值）。

### 1.1.3 代码实战：使用Python/SQL模拟分组与测算

理解底层逻辑的最好方式是亲自动手实现。下面展示如何使用Python的Pandas库来模拟一个简化的DRG/DIP分组与权重测算过程。

#### Python 模拟 DRG/DIP 分组与权重计算

```
import pandas as pd
import numpy as np

# 1. 构造虚拟出院病案首页数据
np.random.seed(42)
n_patients = 10000

# 模拟主诊断编码、主操作编码、年龄、是否伴有并发症(CC_MCC)、住院总费用
data = {
    'patient_id': range(1, n_patients + 1),
    'primary_diag': np.random.choice(['I21.9', 'J18.9', 'K35.8', 'O80.9', 'C34.9'], n_patients),
    'primary_procedure': np.random.choice(['None', '36.10', '47.09', '74.1', '32.29'], n_patients),
    'age': np.random.normal(55, 15, n_patients).clip(0, 100).astype(int),
    'has_cc_mcc': np.random.choice([0, 1], n_patients, p=[0.7, 0.3]),
    'total_cost': np.random.lognormal(mean=9, sigma=0.8, size=n_patients) # 费用通常呈对数正态分布
}

df_patients = pd.DataFrame(data)
```

```

# 根据临床逻辑微调费用：有手术和有并发症的费用更高
df_patients.loc[df_patients['primary_procedure'] != 'None', 'total_cost'] *= 2.5
df_patients.loc[df_patients['has_cc_mcc'] == 1, 'total_cost'] *= 1.5

print("病案首页明细示例：")
print(df_patients.head())

# =====
# 模拟 DIP 分组逻辑（主诊断 + 主操作）
# =====
print("\n--- DIP分组与分值计算 ---")
# 1. 创建DIP组名：主诊断_主操作
df_patients['dip_group'] = df_patients['primary_diag'] + "_" + df_patients['primary_procedure']

# 2. 计算各DIP组的平均费用与病例数
dip_stats = df_patients.groupby('dip_group')['total_cost'].agg(['mean', 'count']).reset_index()

# 3. 计算全样本平均费用作为基准（Base Cost）
base_cost = df_patients['total_cost'].mean()

# 4. 计算DIP分值（RW / Score）
dip_stats['dip_score'] = dip_stats['mean'] / base_cost

print(f"全样本基准费用：{base_cost:.2f}")
print("DIP病种分值字典（部分）：")
print(dip_stats.sort_values('count', ascending=False).head(10))

# =====
# 模拟简化的 DRG 分组逻辑（ADRG -> 分层）
# =====
print("\n--- DRG分组与权重计算 ---")
def assign_adrg(row):
    """一个极简的ADRG映射逻辑"""
    if row['primary_diag'] == 'I21.9' and row['primary_procedure'] == '36.10':
        return 'F11' # 冠脉搭桥
    elif row['primary_diag'] == 'I21.9':
        return 'F60' # 心梗非手术
    elif row['primary_diag'] == 'K35.8' and row['primary_procedure'] == '47.09':
        return 'G11' # 阑尾切除
    else:
        return 'ZZZ' # 其他组

df_patients['adrg'] = df_patients.apply(assign_adrg, axis=1)

# 细分DRG组：ADRG + CC/MCC分级
df_patients['drg_group'] = df_patients['adrg'] + np.where(df_patients['has_cc_mcc'] == 1, 'A',

```

```
# 计算DRG权重
drg_stats = df_patients.groupby('drg_group')['total_cost'].agg(['mean', 'count']).reset_index()
drg_stats['drg_rw'] = drg_stats['mean'] / base_cost

print("DRG组权重字典（部分）：")
print(drg_stats.sort_values('mean', ascending=False).head())
```

## SQL 实战：DIP病种入组与分值计算

在实际数据仓库环境（如 MySQL, PostgreSQL, Hive）中，分组逻辑往往通过 SQL 实现：

```
-- 1. 构建病种组合字典与平均费用
WITH PatientData AS (
    SELECT
        patient_id,
        primary_diagnosis_code AS diag_code,
        IFNULL(primary_procedure_code, 'NONE') AS proc_code,
        total_expense
    FROM medical_records
    WHERE discharge_date BETWEEN '2025-01-01' AND '2025-12-31'
),
CombinedGroups AS (
    -- DIP分组：诊断+操作
    SELECT
        patient_id,
        CONCAT(diag_code, '-', proc_code) AS dip_packet,
        total_expense
    FROM PatientData
),
GlobalMetrics AS (
    -- 计算全域基准平均费用
    SELECT AVG(total_expense) AS global_avg_expense
    FROM CombinedGroups
)
-- 计算每个DIP组的均费和分值
SELECT
    cg.dip_packet,
    COUNT(cg.patient_id) AS case_count,
    AVG(cg.total_expense) AS packet_avg_expense,
    -- 分值 = 该组均费 / 全局均费
    AVG(cg.total_expense) / MAX(gm.global_avg_expense) AS dip_score
FROM CombinedGroups cg
CROSS JOIN GlobalMetrics gm
```

```
GROUP BY cg.dip_packet
HAVING COUNT(cg.patient_id) >= 15 -- 通常只有达到一定病例数的组合才作为稳定病种
ORDER BY dip_score DESC;
```

## 1.2 医疗技术与难度标尺：深度剖析 CMI

### 1.2.1 CMI 的定义与统计学意义

**CMI（Case-Mix Index，病例组合指数）** 是医院管理中最为核心的指标之一。如果说门诊量、住院人次衡量的是医院的“体量”（Quantity），那么CMI衡量的则是医院产出的“质量与难度”（Quality & Complexity）。

CMI 反映了一家医院、一个科室或一名医生在一段时间内收治患者的平均疾病复杂程度和资源消耗水平。CMI 值越高，意味着收治的疑难重症病例越多，医疗技术难度越大，理论上应获得的医保补偿也越多。

### 1.2.2 CMI 的计算公式与相对权重 (RW)

CMI 的计算非常直观，它是所有出院病例的 **相对权重（RW, Relative Weight）** 的加权平均数。

$$CMI = \frac{\sum_{i=1}^n (\text{某DRG/DIP组病例数}_i \times \text{该组RW}_i)}{\text{总病例数}}$$

其中，RW（相对权重）是医保局在年初基于历史数据制定的“标准难度系数”。如果一个病例分到了 RW=2.5 的组，意味着该病例的预期资源消耗是社会平均水平的 2.5 倍。

**关键洞察：如何提升CMI？** 很多医院管理者存在误区，认为只要多收病人就能提高 CMI。实际上，CMI是“均值”概念。 - 收治大量低权重（RW < CMI当前值）的“轻症”患者，反而会**拉低**整体 CMI。 - 提升 CMI 的核心策略是：**结构优化**。即提高四级手术率、收治高权重病种（RW > CMI当前值）、推行日间手术分流轻症，并确保病案首页的高质量编码（避免高编低漏）。

### 1.2.3 Python 代码实战：科室级 CMI 深度计算与归因分析

以下代码展示了如何基于虚构数据集计算各科室的 CMI，并找出拉低/拉高科室 CMI 的关键病种。

```
import pandas as pd
import numpy as np

# 1. 构造 DRG 权重字典库（模拟医保局下发数据）
```

```

drg_dict = pd.DataFrame({
    'drg_code': ['A', 'B', 'C', 'D', 'E'],
    'drg_name': ['心脏移植', '开颅手术', '复杂肺炎', '普通阑尾炎', '轻度胃肠炎'],
    'rw': [15.5, 8.2, 2.1, 0.8, 0.4]
})

# 2. 构造医院出院流水数据
np.random.seed(100)
records = 5000
dept_choices = ['心胸外科', '神经外科', '呼吸内科', '普外科', '消化内科']

data = {
    'patient_id': range(1, records + 1),
    'dept': np.random.choice(dept_choices, records),
    # 按照科室特色分配不同DRG的概率
    'drg_code': []
}

for d in data['dept']:
    if d == '心胸外科':
        data['drg_code'].append(np.random.choice(['A', 'C', 'D'], p=[0.2, 0.6, 0.2]))
    elif d == '神经外科':
        data['drg_code'].append(np.random.choice(['B', 'C', 'E'], p=[0.3, 0.5, 0.2]))
    else:
        data['drg_code'].append(np.random.choice(['C', 'D', 'E'], p=[0.2, 0.4, 0.4]))

df_hospital = pd.DataFrame(data)
df_hospital = df_hospital.merge(drg_dict, on='drg_code', how='left')

# 3. 计算全院 CMI 和 各科室 CMI
hospital_cmi = df_hospital['rw'].mean()
print(f"全院整体 CMI: {hospital_cmi:.3f}")

dept_cmi = df_hospital.groupby('dept').agg(
    case_count=('patient_id', 'count'),
    total_rw=('rw', 'sum')
).reset_index()
dept_cmi['dept_cmi'] = dept_cmi['total_rw'] / dept_cmi['case_count']
dept_cmi = dept_cmi.sort_values('dept_cmi', ascending=False)

print("\n各科室 CMI 排名:")
print(dept_cmi.to_string(index=False))

# 4. 深入洞察: 归因分析 (以心胸外科为例)
print("\n--- 心胸外科 CMI 归因分析 ---")
thoracic_df = df_hospital[df_hospital['dept'] == '心胸外科']
thoracic_cmi = thoracic_df['rw'].mean()

```

```

thoracic_analysis = thoracic_df.groupby(['drg_code', 'drg_name', 'rw']).size().reset_index(name='cases')
thoracic_analysis['case_ratio'] = thoracic_analysis['cases'] / thoracic_df.shape[0]
# 贡献度 = 病种占比 * 病种RW
thoracic_analysis['cmi_contribution'] = thoracic_analysis['case_ratio'] * thoracic_analysis['rw']
# 比较该病种的RW与科室当前CMI: 若RW < 科室CMI, 说明该病种在“拖后腿”
thoracic_analysis['pull_effect'] = np.where(thoracic_analysis['rw'] > thoracic_cmi, '↑拉高', '↓拉低')

print(f"心胸外科当前CMI: {thoracic_cmi:.3f}")
print(thoracic_analysis[['drg_name', 'rw', 'cases', 'case_ratio', 'cmi_contribution', 'pull_effect']])

```

## 1.3 运营效率的双维评估：时间与费用消耗指数

在DRG/DIP支付体系下，医院不仅要追求技术难度（CMI），更要追求在同等难度下的运营效率。效率越高，意味着在医保给定“一口价”的条件下，医院能结余的利润越多，床位周转也越快。

业界通用的效率评估工具是“双消耗指数”：**费用消耗指数（Cost Consumption Index, CCI）**与**时间消耗指数（Time Consumption Index, TCI）**。这两个指标基于标杆法则，剔除了病例难度的影响，实现了不同科室、不同病种间的横向可比。

### 1.3.1 指数定义与公式

- 费用消耗指数 (CCI)** 衡量医院/科室治疗同类疾病的费用控制水平。
$$CCI = \frac{\sum (\text{本院某DRG组平均费用}) \times \text{本院该组病例数}}{\sum (\text{区域标杆该DRG组平均费用}) \times \text{本院该组病例数}}$$
- $CCI < 1$ ：费用控制优于区域平均水平（处于医保盈利安全区）。
- $CCI > 1$ ：费用超标，存在医保超支倒扣风险。
- 时间消耗指数 (TCI)** 衡量医院/科室治疗同类疾病的床位周转效率（住院天数）。
$$TCI = \frac{\sum (\text{本院某DRG组平均住院天数}) \times \text{本院该组病例数}}{\sum (\text{区域标杆该DRG组平均住院天数}) \times \text{本院该组病例数}}$$
- $TCI < 1$ ：平均住院日短于区域均值，周转快。
- $TCI > 1$ ：压床严重，隐性时间成本高。

### 1.3.2 “双优”散点图矩阵分析

结合这两个指标，我们可以绘制以 (1, 1) 为十字交叉原点的四象限散点图，将科室或病种归入四个象限：- **第一象限 (高耗时, 高耗费)**：双劣区。运营效率极差，重点整改对象，需要临床路径

规范化。- **第二象限 (低耗时，高耗费)**：快周转、高消耗。通常见于大量使用昂贵耗材/靶向药但手术恢复快的外科。需严控药耗占比。- **第三象限 (低耗时，低耗费)**：双优区（黄金象限）。运营效率极高，是医院DRG结余的利润奶牛，应予以资源倾斜。- **第四象限 (高耗时，低耗费)**：慢周转、低消耗。通常见于康复期病人较多、压床严重的内科。需加强出院管理和康复医院转诊。

### 1.3.3 Python 实战：绘制“双维评估”四象限散点图

以下代码使用 `matplotlib` 和 `seaborn` 绘制一张精美的运营效率四象限图。

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 解决中文显示问题（需本地有合适字体）
plt.rcParams['font.sans-serif'] = ['SimHei'] # Windows使用黑体
plt.rcParams['axes.unicode_minus'] = False

# 1. 模拟 20 个临床科室的运营效率数据
np.random.seed(2026)
n_depts = 20
depts = [f"临床科室{i+1}" for i in range(n_depts)]

# 模拟 CCI, TCI 以及 圆点大小代表的 权重总值(Total RW)
data = {
    'Department': depts,
    'CCI': np.random.normal(1.0, 0.15, n_depts), # 均值为1.0
    'TCI': np.random.normal(1.0, 0.2, n_depts),
    'Total_RW': np.random.randint(1000, 10000, n_depts) # 气泡大小
}
df_eff = pd.DataFrame(data)

# 2. 绘制散点图矩阵
plt.figure(figsize=(10, 8))
sns.scatterplot(
    data=df_eff,
    x='TCI',
    y='CCI',
    size='Total_RW',
    sizes=(50, 1000),
    alpha=0.7,
    color='royalblue',
    legend=False
)
```



```

# 3. 添加十字标尺（基准线 1.0）
plt.axvline(x=1.0, color='red', linestyle='--', linewidth=1.5)
plt.axhline(y=1.0, color='red', linestyle='--', linewidth=1.5)

# 4. 标注文本
for i in range(df_eff.shape[0]):
    # 仅标注偏离中心较远的科室
    if abs(df_eff['CCI'][i] - 1) > 0.1 or abs(df_eff['TCI'][i] - 1) > 0.15:
        plt.text(
            df_eff['TCI'][i] + 0.02,
            df_eff['CCI'][i],
            df_eff['Department'][i],
            fontsize=9
        )

# 5. 添加象限文字说明
plt.text(0.7, 1.25, "第二象限\n快周转、高消耗\n(重点控费)", fontsize=12, color='darkred', ha='c')
plt.text(1.3, 1.25, "第一象限\n慢周转、高消耗\n(双劣区-预警)", fontsize=12, color='darkred', ha='c')
plt.text(0.7, 0.75, "第三象限\n快周转、低消耗\n(双优区-利润奶牛)", fontsize=12, color='darkgreen', ha='c')
plt.text(1.3, 0.75, "第四象限\n慢周转、低消耗\n(清理压床)", fontsize=12, color='darkblue', ha='c')

plt.title("临床科室运营效率双维评估矩阵 (CCI vs TCI)", fontsize=16, pad=20)
plt.xlabel("时间消耗指数 (TCI)", fontsize=12)
plt.ylabel("费用消耗指数 (CCI)", fontsize=12)
plt.grid(True, alpha=0.3)
plt.xlim(0.5, 1.5)
plt.ylim(0.6, 1.4)
plt.tight_layout()

# 实际使用中可以用 plt.savefig('efficiency_matrix.png') 保存图片
print("四象限散点图绘制逻辑执行完毕。注意观察处于第三象限的科室。")

```

## 1.4 组内同质性与特病单议：变异系数（CV）的统计学应用

DRG/DIP支付的底层假设是：分入同一组的病例，其资源消耗应具有高度的相似性（同质性）。但现实临床情况千变万化，如果某一组内的患者费用天差地别，按平均标准进行“一口价”付费对医院是不公平的，极易导致医院“推诿重患”。

因此，评估分组同质性，并识别出那些费用极度偏高的“异类”（Outliers），是医保运营管理的中中之重。针对这些异类病例，医保局通常设立了“特病单议”（按项目据实结算）的救济通道。

### 1.4.1 变异系数 (CV) 的统计学理论

如何衡量一组数据的离散程度？最常用的指标是标准差（Standard Deviation）。但在比较不同费用的DRG组时，直接用标准差并不合适。例如：- DRG组A（感冒）：均费 3000，标准差 1000。- DRG组B（心脏移植）：均费 300,000，标准差 30000。绝对值上看，B组波动极大，但相对其巨大的基数，其稳定性其实远高于A组。

为了消除量纲和均值大小的影响，统计学引入了 **变异系数 (Coefficient of Variation, CV)**： 
$$\text{CV} = \frac{\text{标准差} (\sigma)}{\text{平均值} (\mu)}$$

在医保分组中：- **CV < 0.8 或 CV < 1.0**（各省标准不同）被认为是组内同质性良好，适合实行打包付费。- **CV 过高**：说明该组异质性大，可能存在严重高倍率病例，或该组的分类逻辑本身存在缺陷。

### 1.4.2 识别异常值 (Outliers) 与特病单议策略

对于组内的个体病例，如何界定其是否属于极端高耗费的异常病例？常用的统计学方法包括：

1. **经验法则 (3-Sigma 准则)**：若费用服从正态分布，超出  $\mu + 3\sigma$  的病例为异常。但医疗费用往往呈右偏分布（少数人花费极高），不严格服从正态分布。
2. **箱线图法则 (IQR 准则)**：这是更稳健 (Robust) 的非参数方法。
3. 计算上四分位数 (Q3) 和下四分位数 (Q1)。
4. 四分位距  $IQR = Q3 - Q1$ 。
5. 异常高值阈值通常设定为  $Q3 + 1.5 \times IQR$ 。
6. 在很多省份的医保政策中，费用超过DRG均费的 2 ~ 3 倍被称为“高倍率病例”，也是申请“特病单议”的门槛。

对于医院而言，建立自动化的高倍率病例预警机制，及时收集病历证明其资源消耗的合理性（如抢救、使用昂贵特效生命支持设备），及时向上级医保部门发起特病单议申请，是挽回医保亏损的极其重要的一环。

### 1.4.3 Python 实战：CV计算与异常值识别（箱线图）

以下代码模拟了多个DRG组的费用数据，计算它们的变异系数，并使用箱线图逻辑自动筛查出“疑似可申请特病单议”的高倍率病例。

```
import pandas as pd
import numpy as np

# 1. 模拟三个不同特性的DRG组费用数据
np.random.seed(999)
```

```

# 组1: 同质性好 (变异小) - 如单纯性白内障手术
group_1_costs = np.random.normal(loc=6000, scale=800, size=500)

# 组2: 同质性一般 (变异中等) - 如复杂肺炎
group_2_costs = np.random.lognormal(mean=9.2, sigma=0.5, size=500)

# 组3: 同质性差, 含有极高异常值 - 如重症多发伤
group_3_costs = np.concatenate([
    np.random.normal(loc=50000, scale=8000, size=480),
    np.random.normal(loc=250000, scale=20000, size=20) # 20个超高异常值
])

# 组合数据
df_costs = pd.DataFrame({
    'patient_id': range(1, 1501),
    'drg_group': ['DRG-1']*500 + ['DRG-2']*500 + ['DRG-3']*500,
    'total_cost': np.concatenate([group_1_costs, group_2_costs, group_3_costs])
})

# 2. 计算各组的 均值、标准差、变异系数 (CV)
stats = df_costs.groupby('drg_group')['total_cost'].agg(['mean', 'std', 'count']).reset_index()
stats['CV'] = stats['std'] / stats['mean']

print("--- 各DRG组变异系数(CV)评估 ---")
print("判断标准: CV < 0.8 通常认为组内同质性较好")
print(stats.round(2))
# 输出将显示 DRG-3 的 CV 显著偏高

# 3. 异常值(Outliers)识别算法: IQR 箱线图法则寻找"特病单议"潜在对象
def identify_outliers(group_df):
    Q1 = group_df['total_cost'].quantile(0.25)
    Q3 = group_df['total_cost'].quantile(0.75)
    IQR = Q3 - Q1
    # 设定高倍率阈值, 这里使用严格一点的 Q3 + 2*IQR 或者 直接大于均值的3倍
    upper_bound = Q3 + 2 * IQR
    mean_cost = group_df['total_cost'].mean()
    policy_bound = mean_cost * 3 # 模拟某地政策: 超过均费3倍算高倍率

    # 选取同时满足统计学异常和政策异常的病例
    threshold = max(upper_bound, policy_bound)

    outliers = group_df[group_df['total_cost'] > threshold].copy()
    outliers['threshold'] = threshold
    outliers['exceed_ratio'] = outliers['total_cost'] / mean_cost
    return outliers

```

```

# 应用算法查找潜在的特病单议病例
print("\n--- 识别高倍率病例（潜在特病单议清单） ---")
outlier_list = []
for name, group in df_costs.groupby('drg_group'):
    outs = identify_outliers(group)
    if not outs.empty:
        outlier_list.append(outs)

if outlier_list:
    df_outliers = pd.concat(outlier_list)
    df_outliers = df_outliers.sort_values('exceed_ratio', ascending=False)
    print(f"共发现 {len(df_outliers)} 例符合高倍率特征的异常病例，建议启动病案核查并申请特病单议")
    print(df_outliers[['patient_id', 'drg_group', 'total_cost', 'exceed_ratio', 'threshold']].head(5))
else:
    print("未发现严重偏离的高倍率病例。")

```

通过上述数据分析与算法筛选，医院能够化被动为主动，在医保结算前精准锁定那些可能造成重大亏损的极端病例，为医保申诉提供坚实的数据支撑。

## 结语

本模块从医保支付的历史进程切入，深入拆解了DRG/DIP的分组原理与数学模型，并通过代码实战详细推演了CMI（技术标尺）、双消耗指数（效率标尺）以及CV与特病单议（风险预警标尺）的计算与业务应用。掌握这些硬核的底层逻辑，是进行深层次病种分析的前提，也为我们下一模块深入临床科室的实际业务场景打下了坚实的数据科学基础。